OnTarget Alpha: user stories

OnTarget Alpha: user stories and acceptance criteria

Each story is one GitHub issue (issue number = story number). Tests for each issue are written before the code.

Story 1: Create a day-by-day plan from a company and interview date

As a job candidate with a technical interview coming up, I want to enter a company and my interview date and get a day-by-day prep plan, so that I know exactly what to do each day instead of studying at random.

Acceptance criteria

- ☐ POST /plans with a company and a future interview date returns 201 with a plan id.
- ☐ The plan has one day per day until the interview, capped at 14 days. The last day is the day before the interview.
- ☐ Every day has at least one task, and every task starts as not done.
- ☐ Task ids are unique within a plan.
- ☐ The company name is trimmed.
- ☐ A missing, blank, or over-80-character company returns 422.
- ☐ A missing or invalid date, or a date that is today or earlier, returns 422.

Labels: backend, feature · Tests: `tests/test_create_plan.py`

Story 2: Generate company-specific tasks with the LLM

As a job candidate, I want tasks written for my specific company's interview style, so that my prep matches what I will actually be asked.

Acceptance criteria

- ☐ The planner sends the LLM the company name, the number of days and the list of problem ids, and calls it once when the first answer is usable.
- ☐ A valid JSON answer becomes the plan, and the plan's source is "llm".
- ☐ JSON wrapped in code fences or surrounded by extra text is still read.
- ☐ A coding task whose problem_id is in the catalog keeps the link. An unknown problem_id is dropped and the plan still succeeds.
- ☐ The Anthropic client reads ANTHROPIC_API_KEY and fails with a clear error if it is missing.

Labels: backend, llm, feature · Tests: `tests/test_llm_plan.py`

Story 3: Fall back to a default plan when the LLM fails

As a job candidate, I want to still get a plan if the AI service is down or answers badly, so that I am never blocked before an interview.

Acceptance criteria

- ☐ If the LLM raises an error (including a timeout), POST /plans still returns 201 with source "fallback".
- ☐ An empty answer, text that is not JSON, the wrong number of days, a day with no tasks, an unknown task type, or a task with blank text all trigger the fallback.
- ☐ The fallback plan has the right number of days and mentions the company.
- ☐ A failed or unusable answer is retried, up to 3 attempts in total with a short wait between them, before the default plan is used.
- ☐ Every failed attempt is logged.

Labels: backend, llm, reliability · Tests: `tests/test_fallback.py`

Story 4: Handle lesser-known and unusual company names safely

As a candidate interviewing at a small or lesser-known company, I want a full plan even if the AI has never heard of my company, so that the tool works for any employer, not just big tech.

Acceptance criteria

- ☐ Names with letters from any language, digits, spaces and & . , ' + - ( ) ! / are accepted (AT&T, Nestlé, Procter & Gamble).
- ☐ Repeated spaces inside a name are collapsed to one.
- ☐ Names with line breaks or characters such as { } < > ` are rejected with 422, so they cannot inject instructions into the LLM prompt.
- ☐ The prompt tells the LLM to use general software interview prep when it does not know the company.

- [ ] A company the LLM knows nothing about still gets a full plan.

Labels: backend, llm, security · Tests: `tests/test_company_names.py`

Story 5: Save a plan and load it again later

As a job candidate, I want my plan saved so I can come back to it, so that I do not lose my progress when I close the app.

Acceptance criteria

- [ ] GET /plans/{id} returns the same plan that POST /plans returned.
- [ ] An unknown plan id returns 404.
- [ ] Plans are stored in SQLite and survive an app restart (a new app on the same database file).
- [ ] Each plan gets a different id.

Labels: backend, feature · Tests: `tests/test_persistence.py`

Story 6: Mark tasks done and see my progress

As a job candidate, I want to check off tasks and see how far along I am, so that I can tell if I am on track for the interview.

Acceptance criteria

- [ ] Every plan response includes progress with done, total and percent.
- [ ] PATCH /plans/{id}/tasks/{task_id} with {“done”: true} marks the task done and updates progress.
- [ ] Percent is round(100 * done / total), and 0 when total is 0.
- [ ] A task can be marked not done again.
- [ ] The change is saved.
- [ ] An unknown plan or task returns 404. A body without “done” returns 422.

Labels: backend, feature · Tests: `tests/test_progress.py`

Story 7: Skip a day without hurting my progress

As a busy job candidate, I want to skip a day I cannot study, so that one missed day does not make my plan look failed.

Acceptance criteria

- [ ] PATCH /plans/{id}/days/{day_number} with {“skipped”: true} marks the day skipped. Day numbers start at 1.
- [ ] Tasks in skipped days are left out of both done and total.
- [ ] Setting skipped to false brings the tasks back into progress.
- [ ] Skipping every day gives total 0 and percent 0, with no error.
- [ ] An unknown plan or day number returns 404.
- [ ] The change is saved.

Labels: backend, feature · Tests: `tests/test_skip_day.py`

Story 8: Open a coding problem with starter code

As a job candidate, I want to open a coding task and see the full problem with starter code in my language, so that I can practice inside the app.

Acceptance criteria

- [ ] GET /problems lists the seven catalog problems with id, title and difficulty.
- [ ] GET /problems/{id} returns title, difficulty, topics, description, examples and constraints.
- [ ] Every problem has starter code for Python, Java, C++ and JavaScript.
- [ ] An unknown problem id returns 404.
- [ ] Every problem_id used by a coding task in a new plan exists in the catalog.

Labels: backend, feature · Tests: `tests/test_problems.py`

Story 9: Use OnTarget in a web page served by the backend

As a job candidate, I want to enter a company and date in a web page and see, tick off and practice my plan there, so that I can use OnTarget without writing any API calls.

Acceptance criteria

- [ ] Opening / in a browser shows the OnTarget page with a company field, a date field and a Build my plan button.
- [ ] The page uses the same server for plans and problems, with no hard-coded server address and no external scripts.
- [ ] The page shows whether the plan was written by the AI or is the standard fallback plan.

- [ ] Ticking a task or skipping a day updates the progress and is saved through the API.
- [ ] Opening a coding task shows the problem and an editor; code results are simulated in this version.
- [ ] The API routes and /docs still work, and an unknown path returns 404.

Labels: frontend, feature · Tests: `tests/test_ui.py`